

DIGITIZING THE DATA FOR LAWN SQUAD: TRANSFORMING LAWN SQUAD'S OPERATIONS

By,

Ramya Ramesh

Data Science Student, SAIT

TABLE OF CONTENTS

1. Table of Contents:	Error! Bookmark not defined.
<u>2.</u> Introduction	3
<u>3.</u> Project Mission & Objectives	4
<u>3.1</u> Mission	4
<u>3.2</u> Key Objectives	4
<u>4.</u> Entities and their Relationships	5
<u>4.1</u> Relationship between the entities:	5
<u>5.</u> Entity Relationship Diagram	6
<u>6.</u> Key Tables:	7
<u>7.</u> Relationship between entities	9
<u>8.</u> SQL Views	11
<u>8.1</u> View 1 – Service Assignments	11
<u>8.2</u> View 2 – Window Cleaning Services	12
<u>8.3</u> View 3 – Customer Service History	13
<u>9.</u> Implementation in Azure SQL	14
<u>10.</u> Conclusion:	19

INTRODUCTION

Lawn care businesses like **Lawn Squad** rely on efficient scheduling, customer management, and service tracking. However, manual processes lead to errors, inefficiencies, and lost growth opportunities.

This project demonstrates how **data science and database automation** can revolutionize small service-based businesses.

About Lawn Squad

- **Location:** Calgary, AB
- **Services:**
 - Lawn Aeration
 - Power Raking
 - Window Cleaning
 - Fertilization
- **Pain Points:**
 - Paper-based scheduling
 - No centralized customer database
 - Difficulty tracking employee assignments

PROJECT MISSION & OBJECTIVES

Mission

“To design and implement a digital data system for Lawn Squad that replaces manual logs with automated, scalable workflows.”

Key Objectives

- **Digitize**
 - **Digitize Operations - Replace manual logs with an automated system**
- **Automate**
 - **Automate Business Insights - Transform raw data into actionable reports**
- **Optimize**
 - **Optimize Resource Allocation - Use historical data to predict demand spikes and schedule staff/materials efficiently.**
- **Reduce**
 - **Reduce Costs & Errors**

ENTITIES AND THEIR RELATIONSHIPS

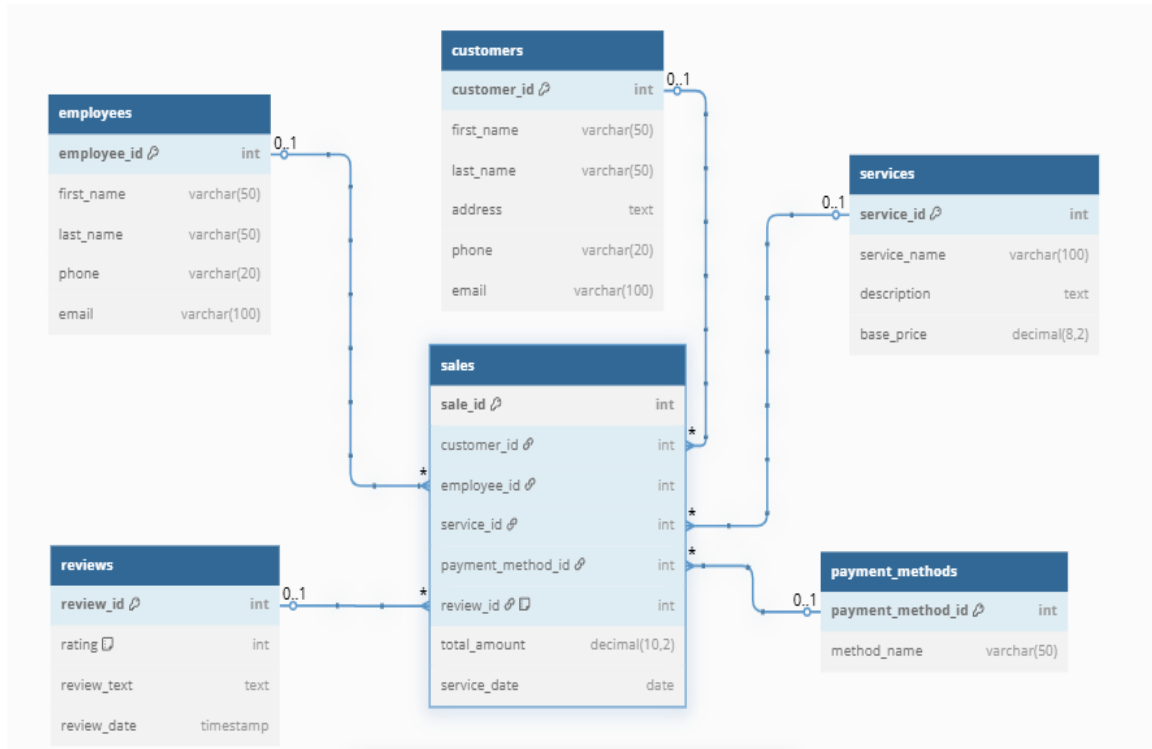
- Customers
- Employees
- Services
- Sales
- Payment Methods
- Reviews

Relationship between the entities:

- Employees - Sales: One-to-Many
- Customers - Sales: One-to-Many
- Sales - Services: Many-to-One
- Sales - Payment Methods: Many-to-One
- Customers - Reviews: One-to-Many

ENTITY RELATIONSHIP DIAGRAM

The ER diagram depicts a star-schema database for Lawn Squads Operations.



1. Core Tables

- **Employees:** Stores staff details (employee_id, name, contact info).
- **Customers:** Tracks client information (customer_id, address, phone, email).
- **Services:** Lists lawn care offerings (service_id, name, description, price).
- **Payment Methods:** Records payment options (payment_method_id, name).

2. Transaction Tables

- **Sales:** Central fact table linking all entities:
 - Foreign keys: customer_id, employee_id, service_id, payment_method_id, review_id
 - Financial data: total_amount, service_date
- **Reviews:** Captures customer feedback (review_id, rating, text, timestamp).

KEY TABLES:

1. Customers table

customers	
customer_id 🔗	int
first_name	varchar(50)
last_name	varchar(50)
address	text
phone	varchar(20)
email	varchar(100)

Purpose: Track client details and service history

2. Employees table

employees	
employee_id 🔗	int
first_name	varchar(50)
last_name	varchar(50)
phone	varchar(20)
email	varchar(100)

Purpose: Holds data about staff members involved in service delivery or sales.

3. Services table

services	
service_id 🔗	int
service_name	varchar(100)
description	text
base_price	decimal(8,2)

Purpose: Contains details about the different services provided to customers.

4. Sales

sales	
sale_id 🔗	int
customer_id 🔗	int
employee_id 🔗	int
service_id 🔗	int
payment_method_id 🔗	int
review_id 🔗	int
total_amount	decimal(10,2)
service_date	date

Purpose: Records transactions involving customers purchasing services.

5. Payment Methods

payment_methods	
payment_method_id 🔗	int
method_name	varchar(50)

Purpose: Lists the types of payment options used in sales transactions.

6. Reviews

reviews	
review_id 🔗	int
rating	int
review_text	text
review_date	timestamp

Purpose: Captures customer feedback and ratings on services received.

RELATIONSHIP BETWEEN ENTITIES

1. Employees → Sales (One-to-Many)

Description:

- One employee can be associated with **multiple sales transactions** (e.g., the same employee handles many lawn services).
- Example: *Employee "Afsal" appears in 5 different sales records for 5 different jobs.*

Why?

Tracks which employees performed which services for accountability and payroll.

2. Customers → Sales (One-to-Many)

Description:

- One customer can have **multiple service bookings** (e.g., the same homeowner books lawn aeration and window cleaning).
- Example: *Customer "Rita McNab" has 3 sales records for recurring services.*

Why?

Maintains complete service history for customer loyalty programs and personalized offers.

3. Sales → Services (Many-to-One)

Description:

- Many sales transactions can reference the **same service** (e.g., 10 customers book "Lawn Aeration").
- Example: *The "Window Cleaning" service appears in 8 different sales records.*

Why?

Standardizes service pricing/descriptions and analyzes service popularity.

4. Sales → Payment Methods (Many-to-One)

Description:

- Many transactions can use the **same payment method type** (e.g., 20 sales paid via "Credit Card").
- Example: *All 5 transactions on Monday used "Bank Transfer."*

Why?

Tracks payment trends and reconciles accounts.

5. Customers → Reviews (One-to-Many)

Description:

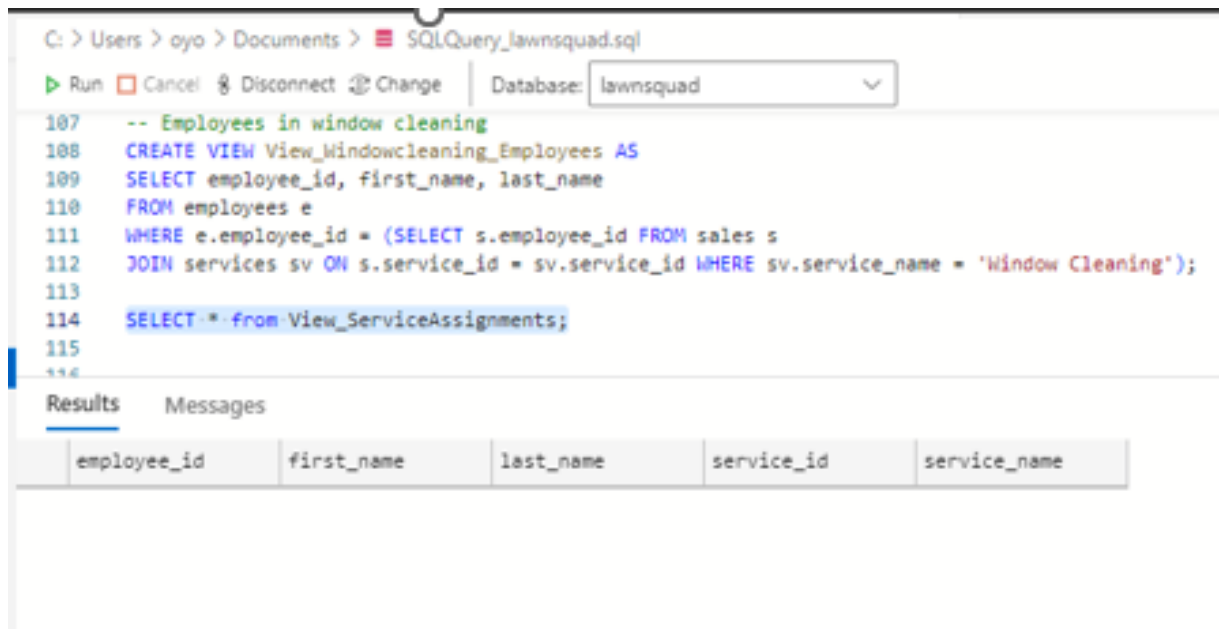
- One customer can leave **multiple reviews** (e.g., after each service).
- Example: *Customer "Bob Johnson" left 2 reviews for different lawn treatments.*

Why?

Improves service quality through feedback and identifies loyal customers.

SQL VIEWS

VIEW 1 – SERVICE ASSIGNMENTS



The screenshot shows a SQL query editor window with the following content:

```
C: > Users > oyo > Documents > SQLQuery_lawnsquad.sql
Run Cancel Disconnect Change Database: lawnsquad
107 -- Employees in window cleaning
108 CREATE VIEW View_Windowcleaning_Employees AS
109 SELECT employee_id, first_name, last_name
110 FROM employees e
111 WHERE e.employee_id = (SELECT s.employee_id FROM sales s
112 JOIN services sv ON s.service_id = sv.service_id WHERE sv.service_name = 'Window Cleaning');
113
114 SELECT * FROM View_ServiceAssignments;
115
```

Below the query editor, there is a 'Results' pane showing the following table structure:

employee_id	first_name	last_name	service_id	service_name
-------------	------------	-----------	------------	--------------

The script creates a view called View_Windowcleaning_Employees, which lists employees involved in "Window Cleaning" services. It joins the employees, sales, and services tables to filter by the service name. Below the view creation, there's a SELECT * FROM View_ServiceAssignments; statement, and the results pane displays columns for employee_id, first_name, last_name, service_id, and service_name.

VIEW 2 – WINDOW CLEANING SERVICES



The screenshot shows the SQL Developer interface with a script editor and a results grid. The script editor contains the following SQL code:

```
113  
114  
115 CREATE VIEW View_ServiceAssignments AS  
116 SELECT s.employee_id, e.first_name, e.last_name, s.service_id, sv.service_name  
117 FROM sales s  
118 INNER JOIN employees e ON s.employee_id = e.employee_id  
119 INNER JOIN services sv ON s.service_id = sv.service_id;  
120  
121 SELECT * FROM View_ServiceAssignments;  
122
```

Below the script editor, the 'Results' tab is active, displaying a table with the following columns:

employee_id	first_name	last_name	service_id	service_name
-------------	------------	-----------	------------	--------------

The view retrieves data from the sales, employees, and services tables, specifically selecting employee_id, first_name, last_name, service_id, and service_name. The script uses INNER JOINS to combine the tables based on matching employee_id and service_id. The final SELECT * FROM View_ServiceAssignments; statement is used to query the view

VIEW 3 – CUSTOMER SERVICE HISTORY

```

138 CREATE VIEW View_CustomerServiceHistory AS
139 SELECT
140     c.customer_id,
141     CONCAT(c.first_name, ' ', c.last_name) AS customer_name,
142     c.email,
143     c.phone,
144     COUNT(s.sale_id) AS total_services_booked,
145     SUM(s.total_amount) AS total_spent,
146     MAX(s.service_date) AS last_service_date,
147     GROUP_CONCAT(DISTINCT sv.service_name SEPARATOR ', ') AS services_used
148 FROM
149     customers c
150 LEFT JOIN
151     sales s ON c.customer_id = s.customer_id
152 LEFT JOIN
153     services sv ON s.service_id = sv.service_id
154 GROUP BY
155     c.customer_id, c.first_name, c.last_name, c.email, c.phone
156 ORDER BY
157     total_spent DESC;
158
159 SELECT * FROM View_CustomerServiceHistory;
160

```

Results Messages

	customer_id	customer_name	email	phone	total_services_booked	total_spent	last_service_date	services_
1	1	Rita McNab	rita@gmail.com	123267546	0	NULL	NULL	NULL
2	2	Janett rorstad	janett@gmail.com	452327578864	0	NULL	NULL	NULL
3	3	bob johnson	bob@gmail.com	34545765824	0	NULL	NULL	NULL
4	4	Justin Hlain	justin@gmail.com	452327578864	0	NULL	NULL	NULL

The view combines data from customers, sales, and services tables to present a summary of each customer's service history. It includes customer ID, full name, email, phone number, total services booked, total amount spent, last service date, and a list of services used. The LEFT JOIN ensures all customers are included, even if they haven't made a purchase. The result set displays these details, though many fields like total_spent, last_service_date, and services_used show NULL, indicating no service history for those customers.

IMPLEMENTATION IN AZURE SQL

```
Create Table employees (  
    employee_id int primary key,  
    first_name varchar(50),  
    last_name varchar(50),  
    phone varchar(20),  
    email varchar(100)
```

```
);
```

```
SELECT * from employees;
```

```
create Table services (  
    service_id int primary key,  
    service_name varchar(100),  
    description text,  
    base_price decimal(8,2)
```

```
);
```

```
SELECT * from services;
```

```
create Table customers (  
    customer_id int primary key,  
    first_name varchar(50),  
    last_name varchar(50),  
    address text,  
    phone varchar(20),  
    email varchar(100)
```

```
);
```

```
SELECT * from customers;
```

```
create Table payment_methods (  
payment_method_id int primary key,  
method_name varchar(50)
```

```
);
```

```
SELECT * from payment_methods;
```

```
--Fact Table
```

```
create Table sales (  
sale_id int primary key,  
customer_id int,  
employee_id int,  
service_id int,  
payment_method_id int,  
total_amount decimal,  
service_date date,  
FOREIGN KEY (employee_id) REFERENCES employees(employee_id),  
FOREIGN KEY (service_id) REFERENCES services(service_id),  
FOREIGN KEY (customer_id) REFERENCES customers(customer_id),  
FOREIGN KEY (payment_method_id) REFERENCES  
payment_methods(payment_method_id)
```

```
);
```

```
SELECT * from sales;
```

```
create Table reviews (  
review_id int primary key,
```

```

customer_id int,
service_id int,
sale_id int,
rating int,
review_text text,
review_date timestamp,
FOREIGN KEY (customer_id) REFERENCES customers(customer_id),
FOREIGN KEY (service_id) REFERENCES services(service_id),
FOREIGN KEY (sale_id) REFERENCES sales(sale_id)

);

SELECT * from reviews;

```

```
--POPULATE TABLE EMPLOYEES
```

```

INSERT INTO EMPLOYEES (employee_id, first_name, last_name, phone, email)
VALUES
(1, 'Ahamed', 'Afsal', '23456784', 'afsal.ahamed@gmail.com'),
(2, 'RAMYA', 'RAMESH', '34874865', 'selenophile63@gmail.com'),
(3, 'dhaval', 'dhaval', '12335246', 'dhaval1245@gmail.com');

```

```
--POPULATE TABLE services
```

```

INSERT INTO services (service_id, service_name, DESCRIPTION, base_price)
VALUES
(11, 'Lawn Aeration', 'Core aeration to relieve soil compaction and improve root growth', 65),
(12, 'Power Raking', 'Removes thatch buildup to enhance water and nutrient absorption', 60),
(13, 'Fertilization', 'Seasonal lawn fertilization to promote healthy green grass', 50),
(14, 'Window Cleaning', 'Interior and exterior window washing for a streak-free shine', 80);

```

```
-- Populate table customers
```



```
INSERT INTO customers (customer_id, first_name, last_name, address, phone, email)
VALUES
```

```
(01, 'Rita','Mcnab', 'jhfasdfgbj843', '123267546', 'rita@gmail.com'),
(02, 'Janett', 'rorstad', 'klsdghaogbabv', '452327578864', 'janett@gmail.com'),
(03, 'bob', 'johnson', 'hgfhvasndvbad', '34545765824', 'bob@gmail.com'),
(04, 'justin', 'blain', 'adjfaiodtgjsfb', '4532245758', 'justin@gmail.com');
```

```
INSERT INTO payment_methods (payment_method_id, method_name)
VALUES
```

```
(001, 'Cash'),
(002, 'Credit Card'),
(003, 'Bank Transfer');
```

```
INSERT INTO sales (sale_id, customer_id, employee_id, service_id, payment_method_id,
total_amount, service_date)
```

```
VALUES
(21, 01, 1, 14, 002, 80, '2023-05-01'), -- Window Cleaning service
(22, 02, 2, 13, 001, 50, '2023-05-02'); -- Fertilization service
```

```
-- Employees in window cleaning
```

```
CREATE VIEW View_Windowcleaning_Employees AS
```

```
SELECT employee_id, first_name, last_name
```

```
FROM employees e
```

```
WHERE e.employee_id = (SELECT s.employee_id FROM sales s
```

```
JOIN services sv ON s.service_id = sv.service_id WHERE sv.service_name = 'Window
Cleaning');
```

```
SELECT * from View_ServiceAssignments;
```

```
CREATE VIEW View_ServiceAssignments AS
```

```
SELECT s.employee_id, e.first_name, e.last_name, s.service_id, sv.service_name
```

```

FROM sales s
INNER JOIN employees e ON s.employee_id = e.employee_id
INNER JOIN services sv ON s.service_id = sv.service_id;

SELECT * from View_ServiceAssignments;

CREATE VIEW View_CustomerServiceHistory AS
SELECT c.customer_id,
CONCAT(c.first_name, ' ', c.last_name) AS customer_name,
c.email, c.phone,
COUNT(s.sale_id) AS total_services_booked,
SUM(s.total_amount) AS total_spent,
MAX(s.service_date) AS last_service_date,
GROUP_CONCAT(DISTINCT sv.service_name SEPARATOR ', ') AS services_used
FROM customers c
LEFT JOIN sales s ON c.customer_id = s.customer_id
LEFT JOIN services sv ON s.service_id = sv.service_id
GROUP BY c.customer_id, c.first_name, c.last_name, c.email, c.phone
ORDER BY total_spent DESC;

SELECT * from View_CustomerServiceHistory;

```

CONCLUSION:

The Lawn Squad database project successfully transforms manual lawn care operations into a streamlined, data-driven system by establishing clear relationships between employees, customers, services, and transactions. By implementing a structured SQL database with One-to-Many and Many-to-One relationships, the system ensures accurate tracking of sales, services, and customer interactions while eliminating inefficiencies like double bookings and payment errors. The automated views provide actionable insights—from employee performance to customer preferences—empowering Lawn Squad to optimize scheduling, improve service quality, and grow sustainably. This foundation not only solves immediate operational challenges but also paves the way for future AI/ML enhancements, such as demand forecasting and dynamic pricing. Ultimately, the project demonstrates how even small businesses can leverage data science to compete smarter, reduce costs, and deliver exceptional customer experiences.